TicketFlow — Track 2

Azure Kubernetes Service (AKS) + Helm Deployment Guide

A step-by-step reference for deploying TicketFlow's three services (API, Identity, Gateway) onto a real Kubernetes cluster using Helm charts, as the demonstration track alongside the primary Azure Container Apps (ACA) deployment.

Project: TicketFlow — .NET 10 / Angular 20 Event Ticketing Platform

Phase: 6 — Container Orchestration (Track 2: AKS)

Related ADRs: ADR-015 (Deployment Strategy), ADR-016 (ACA vs AKS), ADR-017 (AKS Deployment Challenges)

---

1. Architecture Overview

Track 2 deploys the same three containerized services used in the ACA track onto an AKS cluster, using Helm for packaging and an nginx Ingress Controller for external routing. The request flow is:

```
Internet
  -> Azure Load Balancer (public IP)
  -> nginx Ingress Controller (ingress-nginx namespace)
  -> ticketflow-gateway-gateway (Service, ClusterIP)
  -> Gateway pod (YARP reverse proxy)
      -> ticketflow-api-api (Service) -> API pod -> Azure SQL (sqldb-ticketflow-dev)
      -> ticketflow-identity-identity (Service) -> Identity pod -> Azure SQL
        (sqldb-ticketflow-identity-dev)
```

Each service has its own Helm chart, its own Kubernetes Deployment and Service, and (for API and Identity) its own Secret holding database connection strings. The API and Identity services each own a dedicated Azure SQL database — this mirrors the service boundary already established when Identity was extracted from the modular monolith in Phase 5.

2. Prerequisites

Before starting, confirm the following tools are installed and configured:

- Azure CLI (`az`), logged in to the correct subscription
- `kubectl` (installed automatically alongside Azure CLI's AKS tooling, or separately)
- Helm CLI v3+ — install via `winget install Helm.Helm` on Windows
- Docker Desktop, for building and pushing images to ACR
- An existing AKS cluster and ACR, provisioned via Terraform (see Section 3)

> Verify Helm is installed correctly. Note: on Windows, PowerShell must be restarted after installing Helm via winget before the `helm` command is recognized.

```
helm version
```

3. Provisioning the AKS Cluster (Terraform)

The AKS cluster is provisioned via the same `terraform-phase6/main.tf` used for the ACA track, alongside the shared Resource Group, ACR, and Azure SQL Server.

3.1 Choosing a VM size

Before writing the node pool config, confirm the target VM size is both available in the region *and* within your subscription's quota — these are two independent checks.

```
# Check SKU availability in the region
az vm list-skus --location centralus --size Standard_D2s_v3 --output table

# Check subscription quota for the VM family
az vm list-usage --location centralus --output table | Select-String "Standard DSv3"
```

In this project, `Standard_B2s` and `Standard_B2s_v2` were both unusable (unavailable in-region, or zero quota respectively). `Standard_D2s_v3` (2 vCPU / 8GB) was confirmed available with quota headroom and was used for the node pool. See ADR-017, Challenge 1, for the full story.

3.2 AKS cluster Terraform block

```
resource "azurerm_kubernetes_cluster" "main" {
  name                = "aks-${var.project}-${var.environment}-${local.suffix}"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  dns_prefix          = "${var.project}-${local.suffix}"
  tags                = local.tags

  default_node_pool {
    name       = "default"
    node_count = 1
    vm_size    = "Standard_D2s_v3"
  }

  identity {
    type = "SystemAssigned"
  }
}

resource "azurerm_role_assignment" "aks_acr_pull" {
```

```
  principal_id                     =
    azurerm_kubernetes_cluster.main.kubelet_identity[0].object_id
  role_definition_name             = "AcrPull"
  scope                            = azurerm_container_registry.acr.id
  skip_service_principal_aad_check = true
}
```

The `azurerm_role_assignment` is essential — without it, AKS nodes cannot pull images from ACR and every pod will fail with an image pull error.

```
terraform init
terraform plan
terraform apply
```

4. Connecting kubectl to the Cluster

```
az aks get-credentials --resource-group rg-ticketflow-dev-siva06 --name
  aks-ticketflow-dev-siva06

# Verify connectivity
kubectl get nodes
```

Expect one node in `Ready` status, matching the VM size and count configured in Terraform.

5. Installing the nginx Ingress Controller

AKS does not ship with an Ingress Controller by default. Since the Helm charts' `ingress.yaml` templates reference `ingressClassName: nginx`, the ingress-nginx controller must be installed first, or Ingress resources will have no controller to route traffic.

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

helm install ingress-nginx ingress-nginx/ingress-nginx `
  --namespace ingress-nginx `
  --create-namespace
```

This provisions an Azure Load Balancer with a public IP. Allow 1-2 minutes for the IP to be assigned, then confirm:

```
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

Note the `EXTERNAL-IP` on the `ingress-nginx-controller` Service — this is the public entry point for the whole deployment.

6. Helm Chart Structure

Each service has its own standalone Helm chart under a shared `helm/` directory at the project root:

```
ticketflow/
■■■ helm/
    ■■■ ticketflow-api/
    ■   ■■■ Chart.yaml
    ■   ■■■ values.yaml
    ■   ■■■ templates/
    ■       ■■■ deployment.yaml
    ■       ■■■ service.yaml
    ■       ■■■ ingress.yaml
    ■■■ ticketflow-identity/
    ■   ■■■ Chart.yaml
    ■   ■■■ values.yaml
    ■   ■■■ templates/
    ■       ■■■ deployment.yaml
    ■       ■■■ service.yaml
    ■■■ ticketflow-gateway/
        ■■■ Chart.yaml
        ■■■ values.yaml
        ■■■ templates/
            ■■■ deployment.yaml
            ■■■ service.yaml
            ■■■ ingress.yaml
```

Only the Gateway and API charts include an `ingress.yaml` — Identity is intentionally not exposed via Ingress, since it should only be reachable internally via the Gateway, matching its role as an internal-only service in the ACA track.

6.1 Key values.yaml settings

Each chart's `values.yaml` defines the image reference, service type/port, resource limits, and environment variables. Example — `ticketflow-api/values.yaml`:

```
image:
  repository: acrticketflowdevsiva06.azurecr.io/ticketflow-api
  pullPolicy: IfNotPresent
  tag: "latest"

service:
  type: ClusterIP
  port: 80
  targetPort: 8080

env:
  ASPNETCORE_ENVIRONMENT: "Production"
  ASPNETCORE_URLS: "http://+:8080"

secretEnv:
  ConnectionStrings__Default: ""
  ServiceBus__ConnectionString: ""
  IdentityService__BaseUrl: "http://ticketflow-identity-identity"
```

Critical: the keys under `secretEnv` must match the exact configuration key names in the service's own `appsettings.json` — not a guessed or conventional name. .NET's configuration binder silently ignores an environment variable that doesn't match an existing config path; it does not throw an error, it simply falls back to the

default already baked into `appsettings.json` (which, in this project, was LocalDB). Always check the target service's `appsettings.json` directly before naming these keys. See ADR-017, Challenges 5 and 7, for two real incidents caused by this exact mismatch.

7. Helm Release Naming — Avoiding Doubled Resource Names

Templates in this project build resource names as `{{ .Release.Name }}-api-secrets`, `{{ .Release.Name }}-api`, etc. Because each chart's intended release name already contains the service name (e.g. `ticketflow-api`), installing with a matching release name produces a doubled suffix:

```
helm install ticketflow-api .\helm\ticketflow-api
# Produces Service/Secret names like:
#   ticketflow-api-api        (Service)
#   ticketflow-api-api-secrets (Secret)
```

This is not a bug to fix retroactively in this project (Secrets and cross-service URLs are already wired to the doubled names) — it's a naming quirk to be aware of. Always confirm actual resource names with `kubectl get svc` / `kubectl get secrets` rather than assuming the un-doubled name.

8. Creating Kubernetes Secrets

Connection strings and other sensitive values are not stored in `values.yaml` directly — they're injected via Kubernetes Secrets, referenced by name in each Deployment template. These Secrets must be created manually before (or immediately after) the first `helm install`.

```
kubectl create secret generic ticketflow-api-api-secrets `
--from-literal=ConnectionStrings__Default="Server=sql-ticketflow-dev-siva06.database.windows.net;Database=sqld
    Id=sqladmin;Password=<PASSWORD>;Encrypt=True;" `
  --from-literal=ServiceBus__ConnectionString="<SERVICEBUS_CONNECTION_STRING>" `
  --from-literal=IdentityService__BaseUrl="http://ticketflow-identity-identity"

kubectl create secret generic ticketflow-identity-identity-secrets `
--from-literal=ConnectionStrings__IdentityDb="Server=sql-ticketflow-dev-siva06.database.windows.net;Database=s
    Id=sqladmin;Password=<PASSWORD>;Encrypt=True;"
```

Note the different connection string key names: the main API uses `ConnectionStrings__Default`, while Identity uses `ConnectionStrings__IdentityDb` — reflecting each service's own `appsettings.json`. Identity also uses its own dedicated database (`sqldb-ticketflow-identity-dev`) rather than sharing the main API's database, to avoid an EF Core migration collision (ADR-017, Challenge 6).

Retrieve the Service Bus connection string from Terraform output rather than typing it manually:

```
terraform output -raw servicebus_connection_string
```

9. Deploying the Charts

Install the three charts from the project root, in dependency order (Identity and API before Gateway, since Gateway proxies to both):

```
cd C:\Claude\Siva_Projects\ticketflow

helm install ticketflow-identity .\helm\ticketflow-identity
helm install ticketflow-api .\helm\ticketflow-api
helm install ticketflow-gateway .\helm\ticketflow-gateway
```

Check rollout status:

```
kubectl get pods
kubectl get svc
```

All three pods should reach `1/1 Running`. If a pod shows `CreateContainerConfigError`, it usually means a referenced Secret doesn't exist yet, or its key name doesn't match the Deployment spec — recheck Section 8.

9.1 Updating an existing release

After editing a chart's `values.yaml` or templates, apply the change with `helm upgrade`, not a repeated `helm install`:

```
helm upgrade ticketflow-gateway .\helm\ticketflow-gateway

# Force pods to pick up the new config immediately
kubectl delete pod -l app=ticketflow-gateway-gateway
```

A pod that has already crashed with `CreateContainerConfigError` will not automatically retry once the missing dependency (e.g. a Secret) is created — delete the pod manually to force the Deployment controller to recreate it.

10. Configuring the Gateway's Reverse Proxy (YARP)

The Gateway's routing configuration lives in its own `appsettings.json` as a nested `ReverseProxy:Clusters:<clusterId>:Destinations:<destinationName>:Address` structure. To override the destination address per environment, the environment variable name must reproduce that exact hierarchy using double underscores:

```
env:
  ReverseProxy__Clusters__api-cluster__Destinations__main-api__Address:
    "http://ticketflow-api-api"
  ReverseProxy__Clusters__identity-cluster__Destinations__identity-api__Address:
    "http://ticketflow-identity-identity"
```

The cluster ID (`api-cluster`) and destination name (`main-api`) must match the real `appsettings.json` exactly — a fabricated or transposed name will be silently ignored, and YARP will fall back to whatever address is hardcoded in `appsettings.json` for local development (commonly `https://localhost:<port>`), producing a `502 Bad Gateway` once deployed. See ADR-017, Challenge 7.

11. Verifying the Deployment End-to-End

11.1 Internal test (bypasses Ingress/Load Balancer)

```
kubectl run curl-test --image=curlimages/curl --rm --attach --restart=Never -- \
  curl -v http://ticketflow-gateway-gateway/api/events
```

A `200 OK` here confirms Gateway, API, and the database connection are all working — independent of external networking.

11.2 External test (through the public Load Balancer IP)

```
curl.exe http://<EXTERNAL-IP>/api/events
```

If the internal test succeeds but this external test times out from every network you try (not just one), the most likely cause is the Azure Load Balancer's health probe — see Section 12.

12. Fixing Azure Load Balancer Health Probe Failures

By default, AKS configures the Load Balancer's health probe to send `GET /` to the ingress controller's NodePort. If the deployed application has no route mapped to `/` (as is the case here — only `/api/*` is routed), nginx-ingress returns `404`, Azure marks the backend unhealthy, and **silently drops all external traffic** — even though the pod, Service, and internal routing are completely healthy. This is the single most disruptive issue encountered in this track; see ADR-017, Challenge 8, for the full diagnostic story.

Fix — point the probe at nginx's dedicated health endpoint instead:

```
kubectl annotate service ingress-nginx-controller -n ingress-nginx \
  service.beta.kubernetes.io/azure-load-balancer-health-probe-request-path=/healthz \
  --overwrite
```

Allow 1-2 minutes for Azure to reprovision the probe, then retest the external URL from Section 11.2.

Diagnostic checklist if external traffic isn't reaching the cluster:

- Confirm the internal test (11.1) succeeds first — isolates app vs. networking
- Test from two independent networks (e.g. home WiFi and cellular data) to rule out a local firewall
- Check NSG rules allow inbound 80/443 to the Load Balancer's public IP: `az network nsg rule list`
- Check the Load Balancer's health probe configuration and request path: `az network lb probe list`

13. Database Separation Between Services

The API and Identity services must use separate Azure SQL databases, even though both live on the same logical SQL Server. Sharing one database causes an EF Core migration collision the first time both services' migrations run against it (each service defines its own `DbContext` and can create overlapping table names, e.g. `Customers`).

```
resource "azurerm_mssql_database" "identity" {
  name        = "sqldb-ticketflow-identity-dev"
  server_id   = azurerm_mssql_server.main.id
  collation   = "SQL_Latin1_General_CP1_CI_AS"
  sku_name    = "Basic"
  max_size_gb = 2
  tags        = local.tags
}
```

This mirrors the "each service owns its data" principle already established when Identity was extracted from the modular monolith in Phase 5.

14. Tearing Down (Stop Billing)

AKS clusters and their nodes incur cost continuously while running. After capturing screenshots/evidence for the portfolio, tear down the full stack:

```
helm uninstall ticketflow-gateway
helm uninstall ticketflow-api
helm uninstall ticketflow-identity
helm uninstall ingress-nginx -n ingress-nginx

cd terraform-phase6
terraform destroy
```

Note: `terraform destroy` removes the ACR along with all pushed images. If infrastructure is recreated later, all three images must be rebuilt and pushed again before any Container Apps or Kubernetes Deployments can start successfully (see ADR-017, Challenge 2).

15. Quick Command Reference

Task	Command
Connect kubectl to AKS	`az aks get-credentials --resource-group <rg> --name <cluster>`
Install a chart	`helm install <release-name> .\helm\<chart-dir>`
Update a chart	`helm upgrade <release-name> .\helm\<chart-dir>`
View computed values	`helm get values <release-name> -a`
List pods	`kubectl get pods`
Pod logs	`kubectl logs <pod-name>`
Previous crash logs	`kubectl logs <pod-name> --previous`

Force pod recreation	`kubectl delete pod <pod-name>`
Check env vars in a pod	`kubectl exec deploy/<deployment> -- printenv`
Internal connectivity test	`kubectl run curl-test --image=curlimages/curl --rm --attach --restart=Never -`
List Services	`kubectl get svc`
Describe a Service	`kubectl describe svc <service-name>`

For the full list of issues encountered while building this track and their root causes, see **ADR-017: AKS Deployment Challenges and Lessons Learned**.